

Application

EcoRide



Logo Ecoride fait sur Canva

Sommaire :

- Manuel d'utilisation.....2
- Documentation technique.....12
- Gestion de projet.....33
- Charte graphique.....42

Manuel d'utilisation



Image Pixabay

Présentation de l'application :

EcoRide est une application web de covoiturage écoresponsable, permettant à des utilisateurs de proposer des trajets en tant que chauffeur, de réserver des trajets en tant que passager ou de combiner les deux pour ceux qui sont passager-chauffeur.

Chacun de ces trois rôles peuvent visualiser leur espace personnel, changer de rôle et voir les trajets programmés et passés.

Seuls les chauffeurs peuvent rajouter un véhicule et écrire leurs préférences durant le trajet.

Des interfaces spécifiques sont prévues pour les employés dans le cadre de la modération des avis et d'attribution des crédits. Ils peuvent accepter ou refuser un avis et en cas de litiges ils peuvent rétribuer une somme partielle au chauffeur.

L'administrateur possède également son tableau de bord lui permettant de gérer les comptes (suspendre ou rétablir), de créer les comptes employés et également de visualiser des statistiques (nombre de covoiturages par jour, nombre de crédits par jour et le total des crédits de la plateforme).

L'interface met en valeur les trajets réalisés avec des véhicules électriques grâce à la « mention écologique » et intègre un système de paiement par crédits interne à la plateforme.

Suite à un trajet, les passagers sont invités par mail à laisser un avis et une note afin de valider le covoiturage et permettre le règlement du chauffeur.

Connexion à l'application :

Lien de l'application : <https://ecoride-project-9138c168add3.herokuapp.com/>

Identifiants profils :

Administrateur :

Pseudo : « Admin »

Email : « admin.ecoride@ecoride.fr »

Mot de passe : « motdepasse123 »

Employé :

Pseudo : « Employe »

E-mail : « employe@ecoride.fr »

Mot de passe : « motdepasse123 »

Chauffeur :

Pseudo : « José chauffeur »

E-mail : « jose.chauffeur.eco@gmail.com »

Mot de passe : « GreenJose.123 » [identique pour gmail]

Passager :

Pseudo : « José passager »

E-mail : « jose.passager.eco@gmail.com »

Mot de passe : « GreenJose.123 » [identique pour gmail]

Passager-chauffeur :

Pseudo : « José passager chauffeur »

E-mail : « jose.passager.chauffeur.eco@gmail.com »

Mot de passe : « GreenJose.123 » [identique pour gmail]

Les profils José Chauffeur, José Passager et José Passager Chauffeur bénéficient d'un compte gmail pour test la validation d'avis. Les identifiants et mots de passe sont les mêmes.

Les différents parcours possibles :

Utilisateur :

Pour chaque utilisateur, je peux :

Accueil :

Accéder à la page d'accueil en cliquant sur l'onglet « Accueil » dans le menu de navigation pour voir une présentation de la plateforme

Contact :

Si je souhaite accéder aux contacts je peux cliquer sur l'onglet « Contact » dans le menu de navigation.

Mention légale :

Je peux accéder aux mentions légales en cliquant sur le lien en bas à droite de la page et ceux depuis n'importe quelle page.

Visiteur :

Accéder aux covoiturages :

En tant que visiteur, depuis la page d'accueil, je peux me rendre sur l'onglet « Covoiturage » depuis le menu de navigation afin de visualiser les trajets proposés.

Avant même de lancer une recherche, je peux filtrer les trajets par ceux qui ont la mention écologique, par leur prix maximum, par la durée, si le chauffeur accepte les animaux ou par la note du chauffeur.

Ensuite, je peux rentrer une date, une ville de départ, une ville d'arrivée. En appuyant sur « Rechercher », les trajets correspondants à mes critères seront proposés. Si aucun trajet n'est proposé, un bouton s'affiche pour proposer de rechercher par la date la plus proche.

Chaque trajet affiche la photo de profil du chauffeur si disponible, une note, le trajet proposé, le nombre de place disponible, le prix du trajet, la date et l'heure du trajet, si c'est un trajet écologique et deux boutons (« Détails » et « Participer »).

Une fois les résultats affichés, si je souhaite plus d'informations, je peux cliquer sur le bouton « Détails » pour voir :

- Le véhicule utilisé (marque, modèle, énergie),
- Les préférences du chauffeur,
- Les avis des précédents passagers,

Si un trajet m'intéresse, je peux cliquer sur « Participer » mais lorsque je confirmerai le trajet, il me sera proposé de me connecter et je serai redirigé vers la page de connexion (accessible également depuis le menu de navigation).

Connexion / Inscription :

Si j'ai un compte, il me suffit de rentrer mes identifiants et je serai redirigé vers mon espace personnel.

Sinon, je peux cliquer sur le bouton « Pas encore de compte ? Inscrivez-vous ». De là, un formulaire m'invite à rentrer un pseudo, à choisir un rôle, rentrer une adresse email et un mot de passe. Une fois les informations saisies, je peux cliquer sur « S'inscrire ». Je serai redirigé vers le formulaire de connexion où je pourrai me connecter.

Passager :

Mon espace :

Une fois connecté je suis redirigé vers mon espace personnel. En cliquant sur « Changer la photo », je peux télécharger une photo de profil.

En haut à droite je peux voir le nombre de crédits sur mon compte.

En tant que passager, j'ai accès à mes informations personnelles et je peux modifier en cliquant sur « Modifier » face à chaque ligne, mon prénom, mon email, mon mot de passe et je peux changer de type de rôle.

En dessous j'ai un bouton « Voir mes trajets » qui me permet de voir mes trajets programmés et ceux passés.

Mes trajets :

En cliquant sur « Voir mes trajets », deux onglets s'affichent « Mes trajets en cours » qui montre les trajets programmés et « Historique » qui montre les trajets suivis.

Si je souhaite annuler un trajet, dans l'onglet « Mes trajets en cours », à côté de mon trajet réservé, un bouton « Annuler ma réservation » me permet d'annuler ma réservation.

Accéder aux covoiturages :

En tant que passager, depuis l'onglet « Covoiturage » dans le menu de navigation, je peux visualiser les différents trajets proposés.

Avant même de lancer une recherche, je peux filtrer les trajets par ceux qui ont la mention écologique, par leur prix maximum, par la durée, si le chauffeur accepte les animaux ou par la note du chauffeur.

Ensuite, je peux rentrer une date, une ville de départ, une ville d'arrivée. En appuyant sur « Rechercher », les trajets correspondants à mes critères seront proposés. Si aucun trajet n'est proposé, un bouton s'affiche pour proposer de rechercher par la date la plus proche.

Chaque trajet affiche la photo de profil du chauffeur si disponible, une note, le trajet proposé, le nombre de place disponible, le prix du trajet, la date et l'heure du trajet, si c'est un trajet écologique et deux boutons (« Détails » et « Participer »).

Une fois les résultats affichés, si je souhaite plus d'informations, je peux cliquer sur le bouton « Détails » pour voir :

- Le véhicule utilisé (marque, modèle, énergie),
- Les préférences du chauffeur,
- Les avis des précédents passagers,

Si un trajet m'intéresse, je peux cliquer sur « Participer », il me sera demandé une confirmation de mon choix. Suite à cette confirmation, il me sera demandé le nombre de place que je souhaite (par défaut il est proposé 1). Après cette validation, ma réservation est confirmée.

Laisser un avis et une note :

Suite à un trajet, je reçois un email d'Ecoride m'invitant à laisser un avis. En cliquant sur le bouton présent sur l'email, je suis redirigé vers une page.

En haut figure 5 étoiles grisées. En cliquant sur une étoile (par exemple la quatrième), cela me permet de donner une note (donc 4 dans mon exemple).

En dessous figure un champ dans lequel je peux écrire du texte.

Une fois fait, un bouton m'invite à soumettre mon avis.

Une fois l'avis soumis, la note laissée et associée aux autres notes du chauffeur pour en faire une moyenne.

Déconnexion :

Suite à ma connexion, je peux remarquer que l'onglet « Connexion » dans le menu de navigation est devenu « Déconnexion ». En cliquant dessus cela me permet de me déconnecter.

Chauffeur :

Mon espace :

Une fois connecté, je suis redirigé vers mon espace personnel. En cliquant sur « Changer la photo », je peux télécharger une photo de profil.

En haut à droite je peux voir le nombre de crédits sur mon compte.

En tant que passager, j'ai accès à mes informations personnelles et je peux modifier en cliquant sur « Modifier » face à chaque ligne, mon prénom, mon email, mon mot de passe et je peux changer de type de rôle.

En dessous de cette section figure « Informations véhicule ».

Dans cette partie je retrouve la plaque d'immatriculation enregistrée, la date de première immatriculation, le modèle de la voiture enregistrée, la couleur, la marque, le nombre de places, l'énergie utilisée, si j'accepte les fumeurs, les animaux, si je souhaite discuter durant le trajet, si j'écoute de la musique durant le trajet et enfin un dernier point si je veux rajouter une autre préférence. Chacun de ces points est modifiable grâce au boutons « Modifier » situer en face de chaque ligne.

En dessous, deux boutons. « Supprimer ce véhicule » qui permet de supprimer le véhicule enregistré, et « Ajouter un véhicule » qui, si cliqué, m'invite en renseigner plusieurs des critères ci-dessus afin de préremplir la fiche (les préférences restant à remplir au bon vouloir de l'utilisateur).

Sous ces boutons, figure un autre bouton « Suivant » qui permet de visualiser les autres véhicules enregistrés.

Enfin, en bas de page, un bouton « Voir mes trajets » me permet de voir mes trajets programmés, de proposer un trajet et voir les trajets passés.

Mes trajets :

En cliquant sur « Voir mes trajets », trois onglets s'affichent « Mes trajets en cours » qui montre les trajets programmés, « Mes futurs trajets » qui permet de proposer un nouveau trajet et « Historique » qui montre les trajets effectués.

Depuis l'onglet « Mes trajets en cours », je peux annuler un trajet en cliquant sur le bouton correspondant.

Sinon, je peux cliquer sur « Démarrer » lorsque je commence un covoiturage. Une fois arrivée à destination, je peux cliquer sur le bouton « Arrivée ». Le trajet basculera dans l'onglet « Historique ».

Proposer un trajet :

Si je clique sur l'onglet « Mes futurs trajets », depuis ma page personnelle « Mes trajets ». Je peux sélectionner un véhicule enregistré dans mon catalogue, définir une ville de départ, une ville d'arrivée, un jour et une heure, une durée estimée pour le trajet et le prix souhaité par passager. Attention, sur ce point il est bien précisé que 2 crédits seront retenus par la plateforme pour son bon fonctionnement. Enfin, il m'est proposé d'inscrire le nombre de places disponibles pour ce trajet.

Une fois ces champs remplis je peux cliquer sur « Participer »

Déconnexion :

Suite à ma connexion, je peux remarquer que l'onglet « Connexion » dans le menu de navigation est devenu « Déconnexion ». En cliquant dessus cela me permet de me déconnecter.

Passager-chauffeur :

D'une façon plus simple, j'ai les mêmes fonctionnalités que le rôle passager et le rôle chauffeur.

Employés :

Mon espace :

Une fois connecté je suis redirigé vers mon espace personnel duquel je peux changer mes informations personnels (pseudo, email, mot de passe et même le type de compte si nécessaire).

Je peux également modifier ma photo de profil en cliquant sur le bouton « Changer la photo ».

En haut à droite je peux voir que 20 crédits sont inscrits, cela me permet, depuis l'onglet covoiturage, si cela est nécessaire, pour un test ou autre raison, de participer à un covoiturage.

Je possède donc également le bouton « Voir mes trajets ».

Tableau de bord :

En cliquant sur l'onglet « Tableau de bord » depuis le menu de navigation, je peux voir les avis qui sont à valider.

Sous forme de tableau, je peux visualiser l'ID du trajet, la ville de départ, la ville d'arrivée, le chauffeur (son pseudo et son email pour gérer les litiges), le prix du trajet, la date du trajet, le passager qui a laissé l'avis (son pseudo, son email pour gérer les litiges et son ID), le commentaire laissé par le passager, la note laissée par le passager et enfin deux boutons (« Valider » et « Refuser »).

Si je clique sur « Valider », l'avis sera associé au chauffeur et sera visible depuis la page covoiturage (en cliquant sur le bouton « Détails » du trajet associé au chauffeur). Lors de la validation, les crédits (sauf 2 pour la plateforme) sont ajoutés aux crédits du chauffeur. Également, la note est attribuée au chauffeur et une moyenne de l'ensemble des notes le concernant permet d'associer une note au chauffeur.

Si je clique sur « Refuser » (parce qu'il y a un litige), l'avis ne sera pas associé au chauffeur. Suite à ce choix, une proposition s'affiche pour savoir le montant à attribuer, cela peut être 0 (donc pas de crédit) ou une partie. Comme la plateforme conserve 2 crédits il n'est pas possible de valider la totalité du prix.

En haut du tableau, une barre de recherche me permet de filtrer et rechercher un élément spécifique (avec les données actuelles on peut par exemple chercher spécifiquement l'ID du passager).

Il m'est également possible de filtrer par note.

Déconnexion :

Suite à ma connexion, je peux remarquer que l'onglet « Connexion » dans le menu de navigation est devenu « Déconnexion ». En cliquant dessus cela me permet de me déconnecter.

Administrateur :

Mon espace :

Une fois connecté, je suis redirigé vers mon espace personnel duquel je peux changer mes informations personnels (pseudo, email, mot de passe et même le type de compte si nécessaire).

Je peux également modifier ma photo de profil en cliquant sur le bouton « Changer la photo ».

En haut à droite je peux voir que 20 crédits sont inscrits, cela me permet, depuis l'onglet covoiturage, si cela est nécessaire, pour un test ou autre raison, de participer à un covoiturage.

Je possède donc également le bouton « Voir mes trajets ».

Tableau de bord :

En cliquant sur l'onglet « Tableau de bord » depuis le menu de navigation, je peux voir deux graphiques.

Le graphique de gauche montre le nombre de covoiturages par jour, si je mets la souris sur une barre, le nombre précis s'affiche.

Le graphique de droite montre le nombre de crédits cumulé par jour, si je mets la souris sur une barre, le nombre précis s'affiche.

En dessous du graphique « Crédits gagnés par jour », je peux voir le nombre total de crédit perçus depuis la mise en place de la plateforme.

Au-dessus des graphiques, à gauche se trouve un bouton « Créer un compte employé ». En cliquant dessus un formulaire d'inscription apparaît et m'invite à rentrer un pseudo, un mail et un mot de passe. En finalisant avec le bouton « Créer un compte », le compte employé se crée.

En haut à droite du graphique, se trouve le bouton « Suspendre un compte ». En cliquant dessus, il apparaît un tableau sous les graphiques.

On peut voir l'ensemble des comptes existants. De gauche à droite on voit l'ID de l'utilisateur, son pseudo, son email, son rôle, le nombre de crédit de l'utilisateur, sa note et enfin une action. Dans cette action figure un bouton. Soit « Suspendre », soit « Rétablir ». Comme son nom l'indique, « Suspendre » va permettre de suspendre le compte de l'utilisateur (attention à ne pas se suspendre soit même parce que cela empêche la connexion) et « Rétablir » va permettre d'autoriser à nouveau la connexion de l'utilisateur.

Une barre de recherche figure en haut de ce tableau permettant de faire une recherche plus précise suivant n'importe quel critère.

Déconnexion :

Suite à ma connexion, je peux remarquer que l'onglet « Connexion » dans le menu de navigation est devenu « Déconnexion ». En cliquant dessus cela me permet de me déconnecter.

Documentation technique



Image Pixabay

Réflexion initiale sur les technologies utilisées :

L'application EcoRide repose sur une stack technique volontairement légère et maîtrisable

Technologies choisies :

Front End :

- **HTML5 / CSS3 / Bootstrap 5** : structure des pages et mise en forme responsive.
- **JavaScript** : interactivité (fetch), chargement dynamique de page, système de routage dynamique, gestion des événements, création de formulaires dynamiques.
- **Chart.js** : pour l'affichage des statistiques graphiques.

Back End :

-**PHP vanilla** : J'ai choisi PHP vanilla plutôt qu'un framework afin de mieux comprendre le fonctionnement natif du langage, maîtriser entièrement le traitement des requêtes HTTP, la gestion des sessions, des échanges JSON et l'accès aux bases de données. Ce choix me permet de conserver un contrôle total sur la logique applicative et de construire une architecture claire adaptée au projet d'examen, tout en évitant la complexité supplémentaire qu'imposerait un framework complet.

L'application est organisée selon une structure inspirée du modèle MVC simplifié. Les contrôleurs back-end PHP reçoivent les requêtes envoyées par le contrôleur front-end JavaScript via `fetch()`, vérifient la méthode HTTP, contrôlent les droits utilisateur et appliquent les règles de sécurité avant d'appeler les fichiers du modèle. Cette séparation permet de distinguer clairement la gestion de l'expérience utilisateur côté navigateur et la sécurisation des traitements côté serveur.

-**PDO (PHP Data Objects)** est utilisé pour l'accès aux bases de données MySQL. Il permet d'exécuter des requêtes préparées avec paramètres bindés, limitant fortement les risques d'injection SQL et assurant une gestion fiable des connexions. Toutes les opérations sensibles utilisent ce mécanisme sécurisé.

-**POO** : Afin d'améliorer la maintenabilité, la lisibilité et l'évolutivité du projet, une structuration partielle en Programmation Orientée Objet a été introduite. La classe `Covoiturage` a notamment été créée pour encapsuler les informations liées à un trajet (départ, arrivée, horaire, état, prix, véhicule, etc.) et fournir des méthodes métier spécifiques, comme la détection d'un trajet écologique selon le type de véhicule.

Cette classe est utilisée dans le fichier `get_covoiturages.php` afin de transformer les résultats SQL en objets puis en tableaux via une méthode `toArray()`, facilitant ainsi le retour des données au format JSON vers le front-end. Elle est placée dans un dossier dédié `Classes` et importée via `require_once`. Par contrainte de temps dans le cadre de l'examen, seule cette fonctionnalité a été structurée en POO, mais cette organisation pourrait être étendue dans une version future aux autres entités comme `Utilisateur` ou `Réservation`.

Bases de données :

- **MySQL (relationnelle)** : pour la gestion des données principales : utilisateurs, covoiturages, véhicules, réservation, préférences.
- **MongoDB (NoSQL)** : pour logs de la plateforme (avis, crédits, suspensions...) et me baser sur ces logs pour les statistiques.

Déploiement :

- **Heroku** : hébergement du site PHP avec base de données MySQL (via **JawsDB**) et MongoDB (via **Atlas**).
- **Composer** : gestion des dépendances PHP (phpmailer, mongodb, dotenv).
- **Docker** : conteneurisation du projet avec **Docker** et **Docker Compose** pour un déploiement en local.

Configuration de l'environnement de travail :

Local :

- **XAMPP** : Apache + MySQL pour le serveur local et phpMyAdmin pour la gestion de la base de données.
- **MongoDB Compass** : avec l'ensemble des collections logs NoSQL.
- **Visual Studio Code (VSCode) comme IDE avec les extensions** :
 1. GitGraph pour le suivi des commits,
 2. Live Sass Compiler pour les compilations SCSS en CSS (utilisé pour personnaliser Bootstrap),
 3. PHP Intelephense et PHP Namespace Resolver pour programmer et importer les classes plus facilement,
 4. MongoDB pour VSCode me permettant d'accéder aux collections depuis l'IDE,
 5. PHP Server pour ouvrir un serveur local et PHPUnit pour les tests.
- **Docker (optionnel)** :
 - Sur Windows et Mac, Docker Desktop intègre automatiquement Docker Compose.
 - Sur Linux, Docker Engine et Docker Compose doivent être installés séparément.

Le projet devra donc contenir les fichiers suivants liés à Docker :

1. **Dockerfile** : définit l'image Apache et PHP avec les modules nécessaires (pdo_mysql, etc.).
2. **docker-compose.yml** : orchestre les services web, MySQL et MongoDB avec des volumes persistants.
3. **.dockerignore** : exclut les fichiers inutiles de la construction de l'image.
4. **.htaccess** : permet au routeur JavaScript (SPA) de rediriger toutes les requêtes vers index.php.

Dépendances installées via Composer :

- **phpmailer/phpmailer (v6.9)** : pour l'envoi de mails,
- **mongodb/mongodb (v1.20)** : client MongoDB natif PHP,
- **vlucas/phpdotenv (v5.6)** : gestion sécurisée des variables d'environnement.

Environnement distant :

- **Git via GitHub** : pour versionner mon projet, travailler sur différentes branches (main, dev, feature/*) et revenir sur des versions précédentes quand nécessaire.
- **Heroku** : déploiement de l'application (connecté à GitHub).
- **JawsDB MySQL** : add-on de Heroku pour la base de données relationnelle.
- **MongoDB Atlas** : base de données NoSQL distante pour les logs.
- **MySQL Workbench** : interface pour gérer JawsDB MySQL

Schémas MCD de la base de données relationnelle

(réalisés sur dbdiagram.io)



Diagramme de classe (via draw.io) :

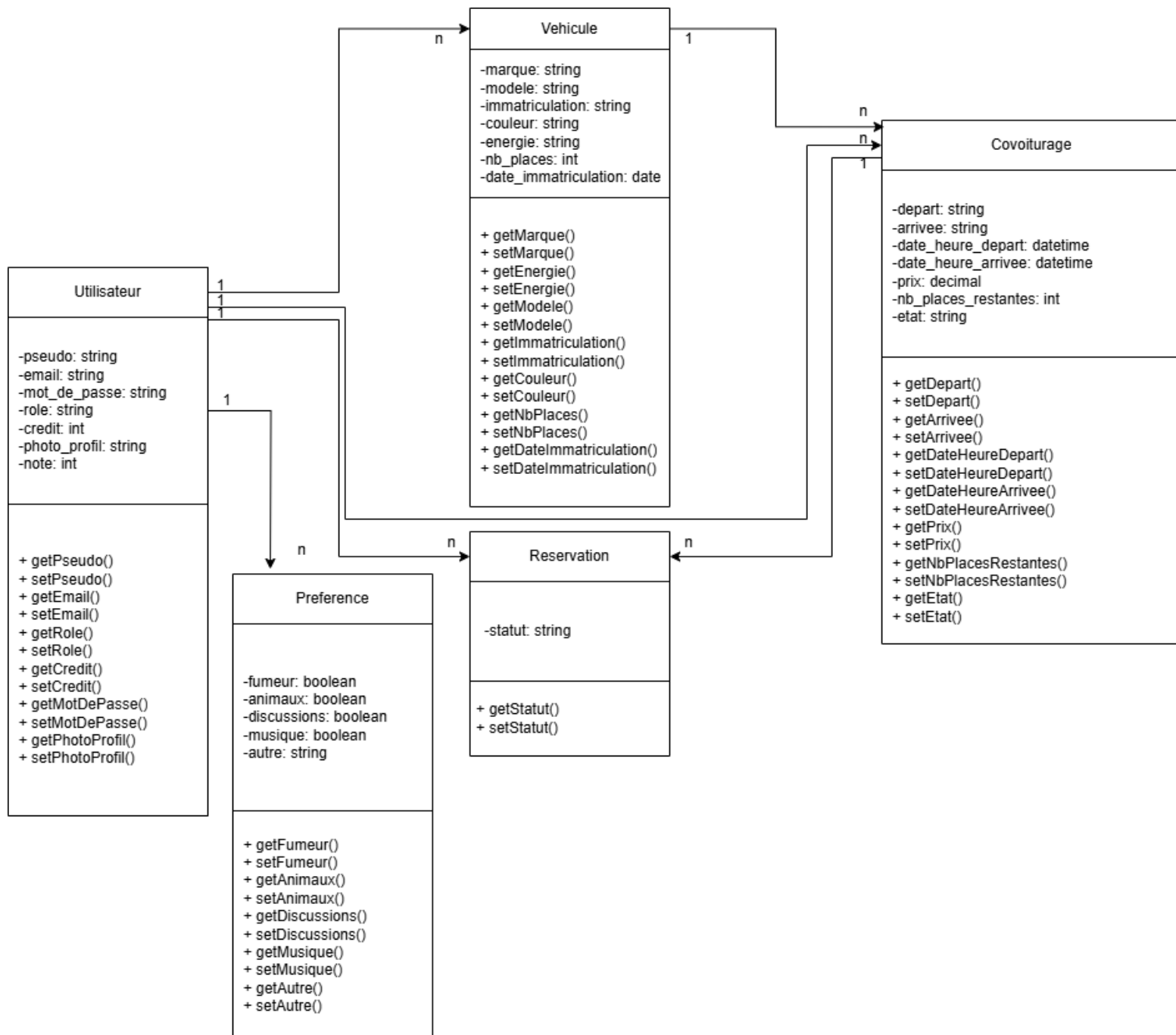
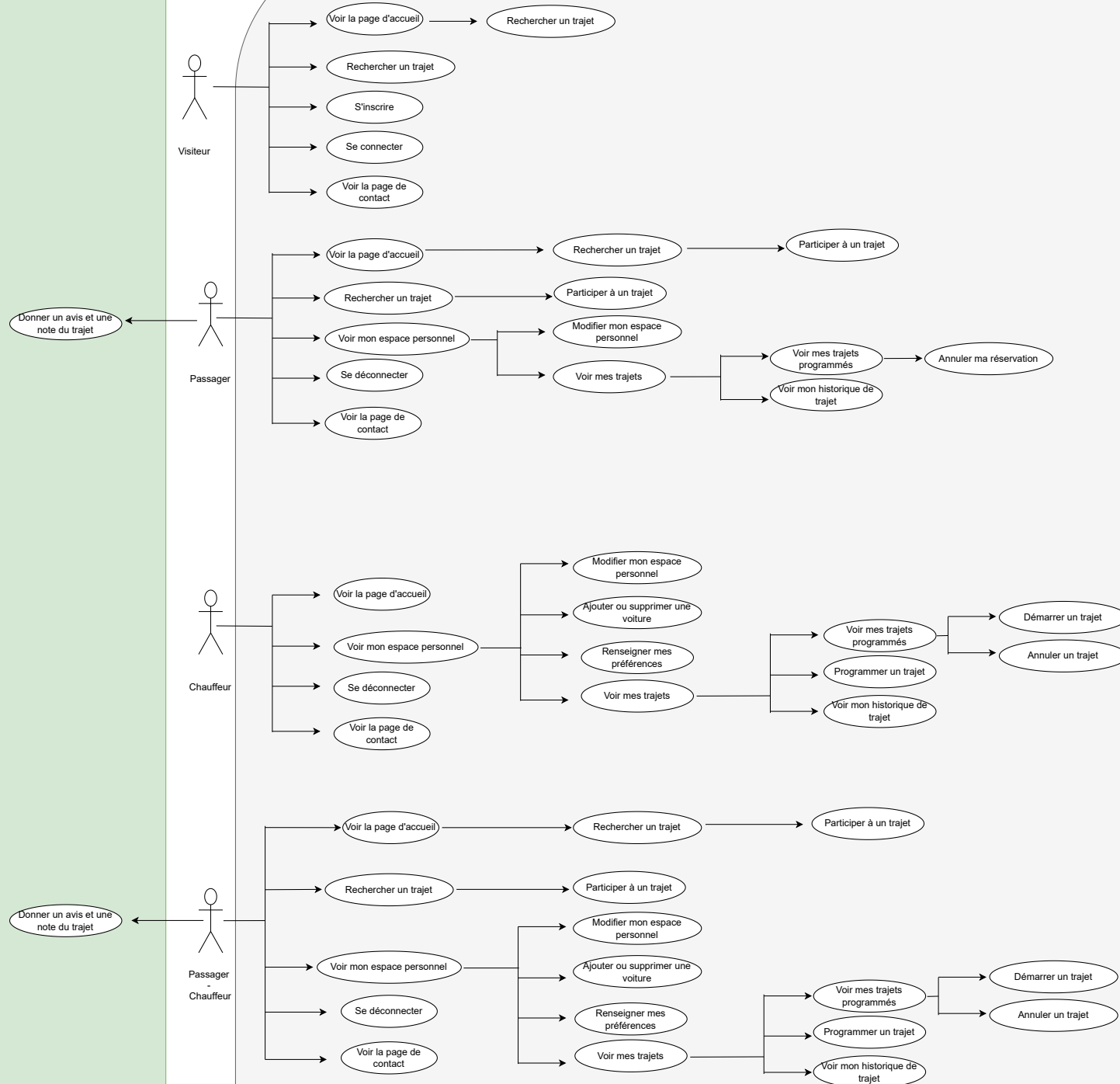


Diagramme d'utilisation via draw.io (insertion du PDF page suivante):

Email

Application Ecoride



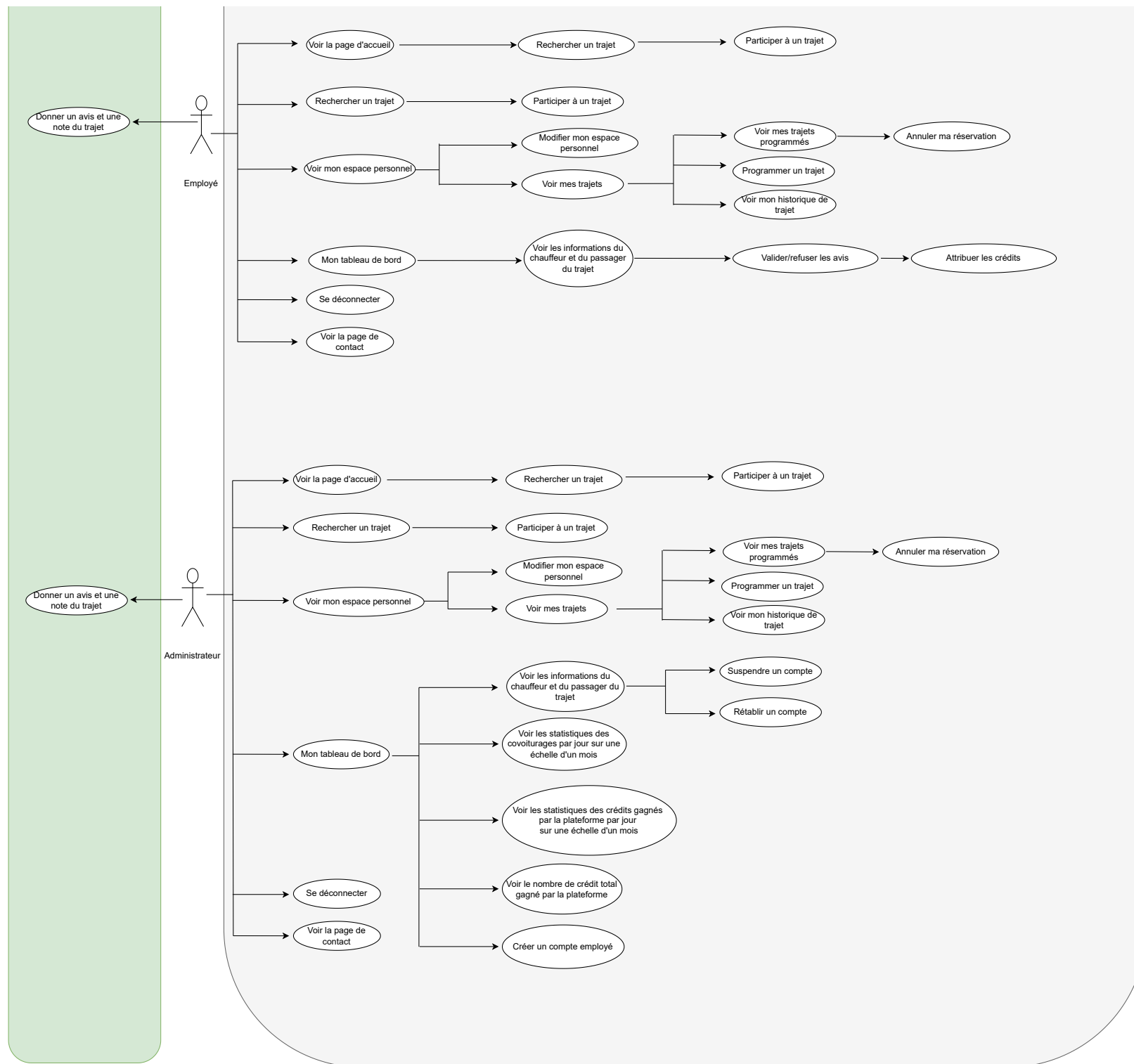


Diagramme de séquence via draw.io (insertion du PDF page suivante) :



Employé

Validation / Refus avis

Interface Vue

espace_employe.php

Controleur Front

employe.js

Controleur Back

gestion_avis_controller.php

Modèle

gestion_avis.php

Base de données

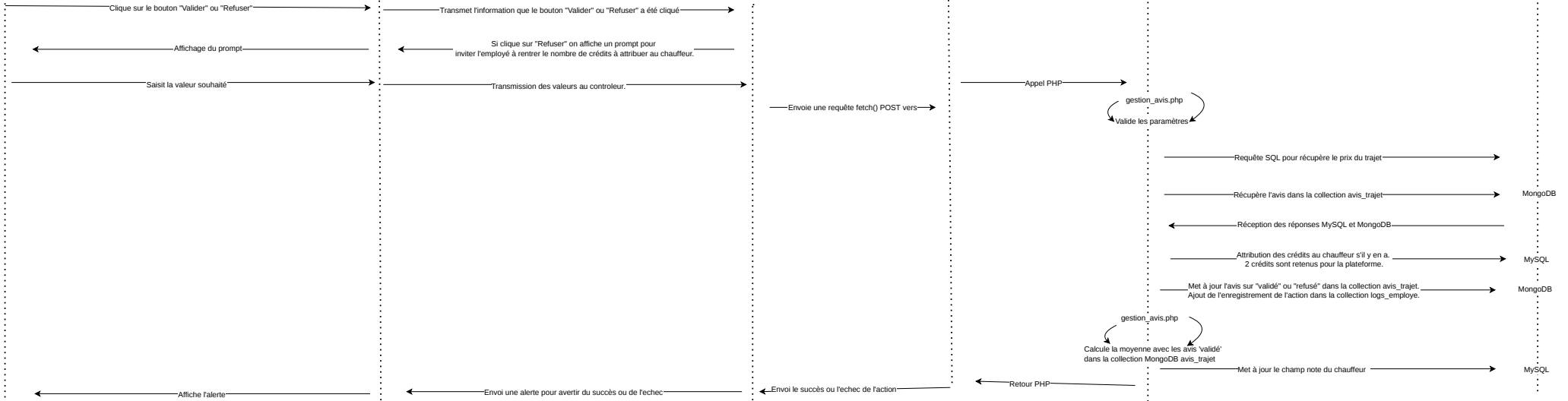
MySQL

MongoDB

MySQL

MongoDB

MySQL





Ajouter un véhicule

Utilisateur

Interface Vue

Controleur Front

Controleur Back

Modèle

Base de données

userSpace.php

userSpace.js

add_vehicule_controller.php

add_vehicule.php

MySQL

MongoDB

Clique sur le bouton « Ajouter un véhicule »

Informe le controleur que "Modifier" a été cliqué avec l'editField associé

Affiche une série de prompt et select pour saisir (marque, modèle, immatriculation, couleur, nombre de places, énergie (dropdown), date immatriculation (input type="date"))

Saisit les valeurs

Envoie une requête fetch() POST vers

Vérifie les bon type de valeur

Appel PHP

Demande l'exécution de la requête
INSERT INTO vehicules (...) VALUES (?, ?, ?, ?, ?, ?, ?)

Confirme l'ajout

Enregistre l'action de réservation dans la collection logs

Affiche le véhicule

Affiche une alerte de succès
Recharge la page

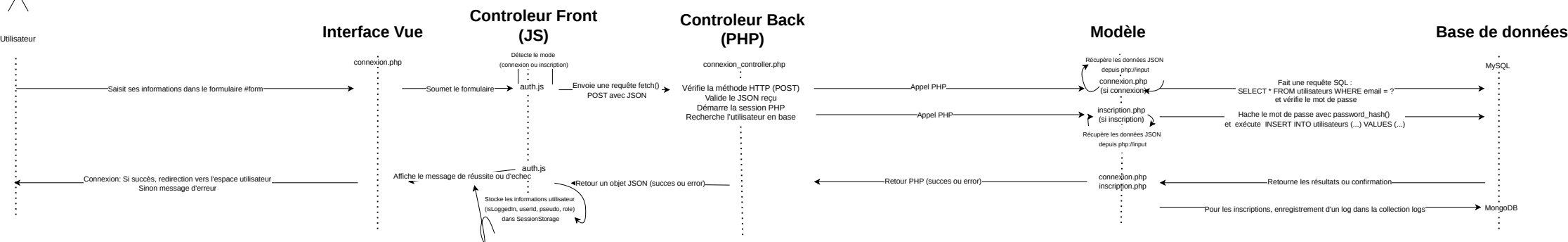
Retourne en JSON le succès

Retour PHP



Utilisateur

Connexion / Inscription





Gestion des trajets chauffeur

Utilisateur Chauffeur

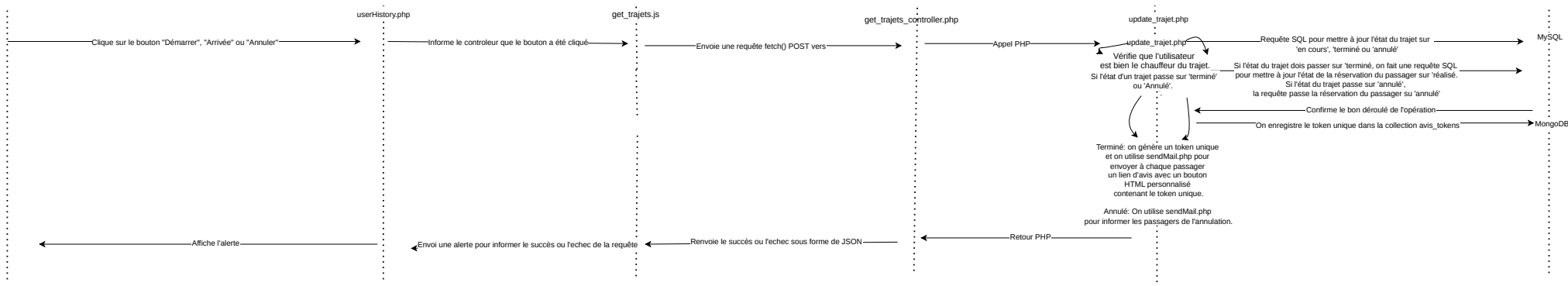
Interface Vue

Controleur Front

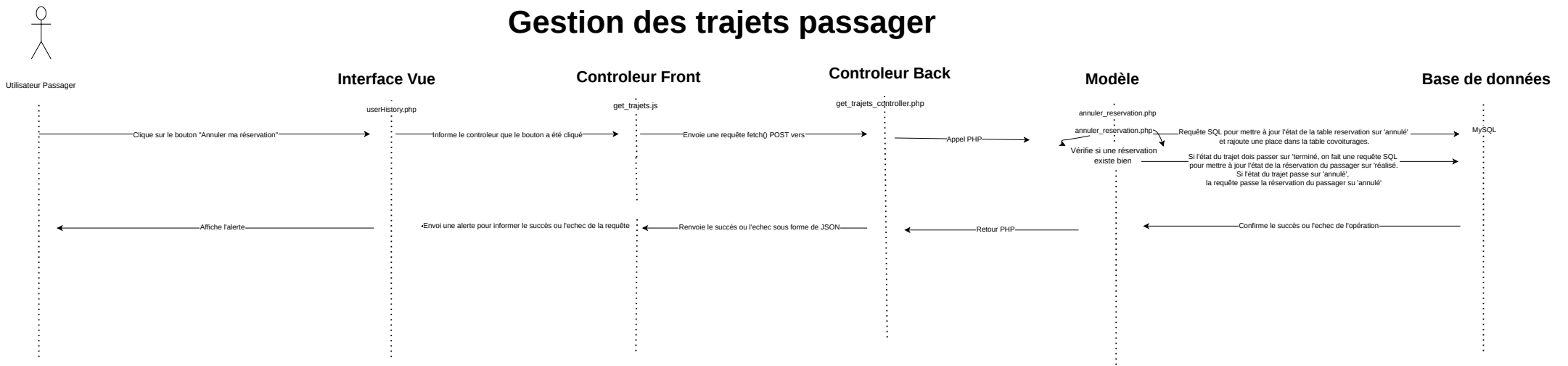
Controleur Back

Modèle

Base de données

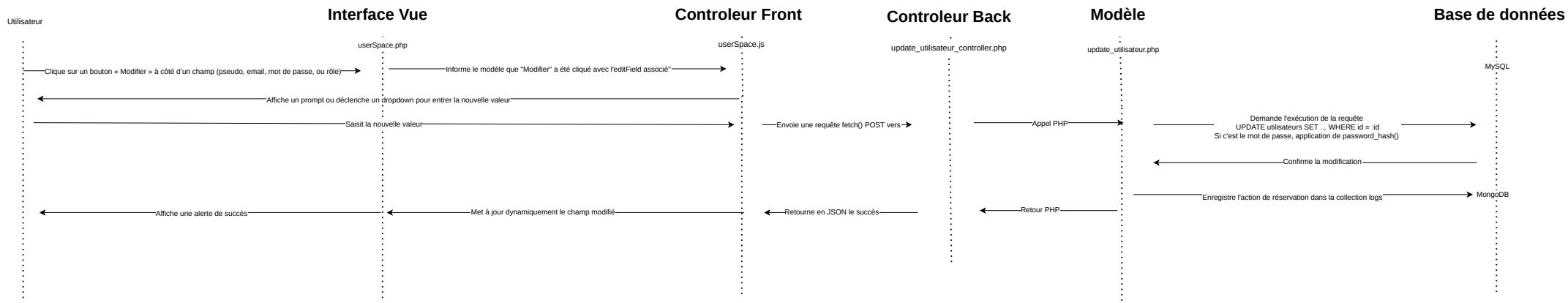


Gestion des trajets passager



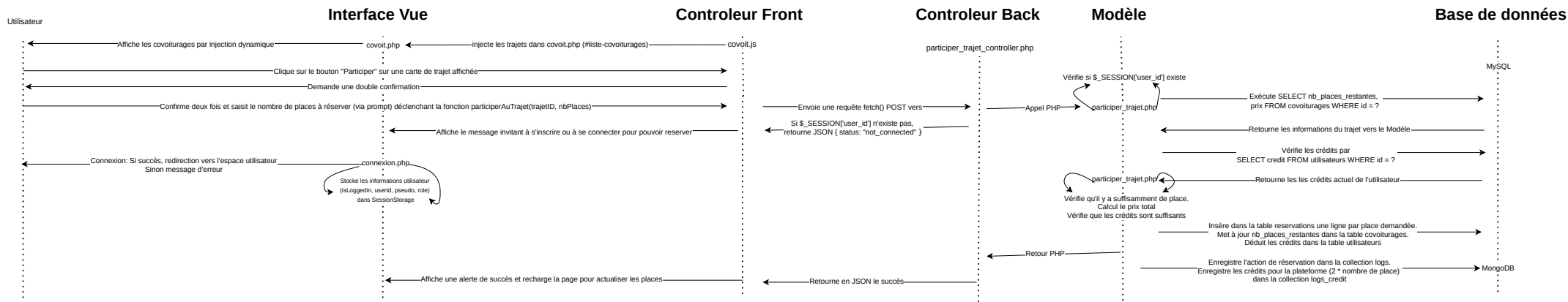


Modifier mon espace





Participer à un trajet





Programmer un trajet

Utilisateur

Interface Vue

userHistory.php

Controleur Front

new_trajet.js

Controleur Back

get_user_controller.php

Modèle

get_user.php

Base de données

MySQL

MongoDB

Accède à l'onglet "Mes futurs trajets" dans l'espace personnel

Affiche le formulaire pour proposer un nouveau trajet

Clique sur la sélection de véhicule du catalogue et remplit les champs du formulaire

Informe le controleur que la page est chargée

Propose les véhicules sous forme d'option dans le dropdown

Une fois le bouton "Proposer" cliqué, les valeurs sont envoyés au controleur

Affiche une alerte de succès
Recharge la page

Envoie une requête fetch() POST vers

Renvoie les véhicules sous forme de JSON

new_trajet.js
Envoie une requête fetch() POST vers
Vérifie que tous les champs
sont renseignés et
que les prix est supérieur à 2

Retourne en JSON le succès

Appel PHP

add_trajet.php
Récupère les données en JSON.
Vérifie l'authentification avec \$_SESSION['user_id'].
Calcule l'heure d'arrivée grâce à l'intervalle.

Retour PHP

Requête SQL pour récupérer l'ensemble des véhicules
en lien avec l'ID de l'utilisateur

Récupère les véhicules associés à l'utilisateur

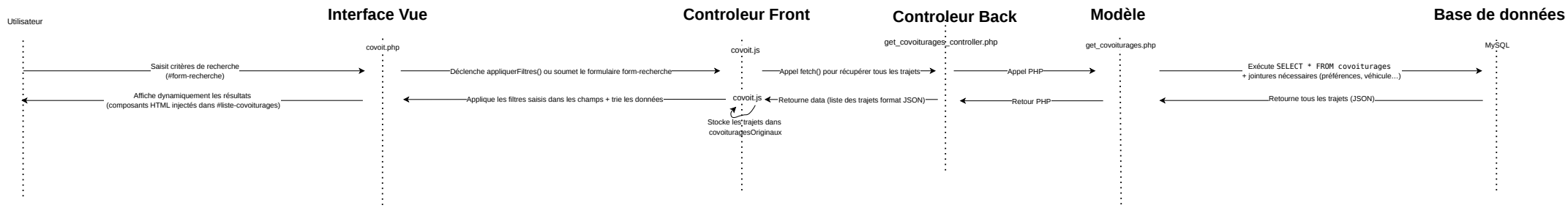
Requête SQL pour insérer le trajet dans la table covoiturages

Renvoie une confirmation

Enregistre l'action de réservation dans la collection logs



Rechercher un trajet





Administrateur

Suspendre / Rétablir compte

Interface Vue

Contrôleur Front

Contrôleur Back

Modèle

Base de données

espace_admin.php

admin.js

suspendre_utilisateur_controller.php

suspendre_utilisateur.php

MySQL

MongoDB

MongoDB

MySQL

Clique sur le bouton "Suspendre" ou "Rétablir"

Informe le contrôleur que le bouton a été cliqué

Envoie une requête fetch() POST vers

Appel PHP

Requête SQL pour vérifier la note de l'utilisateur
(SELECT note FROM utilisateurs WHERE id = ?)

Réception de la note

suspendre_utilisateur.php
Si la note est "-1",
on rétablit le compte.
Sinon, on le suspend.

Suspension du compte (requête SQL): on met la note sur -1
(les comptes avec une note -1 sont bloqués lors de la connexion).
On met les réservations de l'utilisateur sur 'annulé'.
Le nombre de place des covoiturages concernés
par ces réservations annulées sont augmentés de 1.
On annule les covoiturages programmés si c'est un chauffeur.

Confirmation de l'action

On insère dans la collection logs_suspendu
l'ID de l'utilisateur et sa note initiale.

Rétablir le compte: la note est déjà sur -1.
On recherche dans la collection logs_suspendu
la note initiale associée à l'ID de l'utilisateur.

Réception de la note

Requête SQL pour mettre à jour la note de l'utilisateur

Confirmation de l'action

Retour PHP

Affiche l'alerte

Envoi une alerte pour avertir du succès ou de l'échec

Envoi le succès ou l'échec de l'action

Déploiement de l'application (Heroku) :

Préparation GitHub :

- Fusion de la branche de développement avec la branche principale (main)Création du dépôt, branche main, commits progressifs.
- .gitignore configuré pour exclure /vendor/, .env et Modele/data.

Configuration de Heroku :

- Création du compte.
- Création du nom de l'application.
- Dans « Deploy », connexion de Heroku avec mon compte GitHub grâce à mon lien git.
- Dans « Settings », après avoir cliqué sur « Reveal Config Vars », j'ai saisi les variables d'environnements (SMTP* pour les emails, MONGO_URI pour MongoDB Atlas, JAWSDB_URL pour la connexion avec la base de données JawsDB MySQL, DB* pour les paramètres de la base de données JawsDB MySQL et BASE_URL qui est utilisé pour le bouton intégré aux emails que les passagers reçoivent pour laisser un avis auquel je rattache un token unique).

Ajout des addons et outils externes :

- JawsDB MySQL : Dans Heroku, onglet « Ressources » j'ai ajouté l'addon « JawsDB MySQL ». Dans cette plateforme l'URL à copier dans la variable d'environnement était présent. Je l'ai donc rajouté dans la partie « Settings » / « Reveal Config Vars ».
- Installation de MySQLworkbench pour gérer la base de donnée JawsDB MySQL.
- MongoDB Atlas : J'ai créé une base de données eco_ride. J'ai copié URI de la connexion sécurisée que j'ai collé dans les variables d'environnement JawsDB MySQL. Enfin, j'ai autorisé les adresses IP à se connecter à MongoDB Atlas.

Base de données :

- Dans MySQLworkbench, connecté à JawsDB MySQL, j'ai importé ma base de données locale nommé « create_database.sql » (présent sur le dépôt GitHub).
- MongoDB : même si les collections se créent automatiquement par le code, j'ai fait la connexion entre MongoDB Atlas avec MongoDB Compass et j'y ai transféré mes collections pour avoir un historique.

Déploiement :

- Dans « Deploy », j'ai cliqué sur « Deploy Branch » configuré sur la branche git « main ». Lors des corrections, j'ai également déployé la branche « dev » et « bugfix/* » pour faire des tests.

-J'ai adapté mon code pour qu'il fonctionne aussi bien en local que déployé avec Heroku.

-Une fois la branche main déployé, j'ai effectué un test complet du site via <https://ecoride-project-9138c168add3.herokuapp.com/>

Les collections MongoDB utilisées :

- « avis_tokens » : Cette collection contient le token unique du passager et permet de vérifier si un avis à déjà été laissé.
- « avis_trajet » : Contient tous les avis, validés et refusés.
- « logs » : Contient les actions des utilisateurs (véhicules rajoutés, nouveaux membres inscrits, modification sur les comptes etc..)
- « logs_credit » : Contient les crédits retenus par la plateforme à chaque reservation.
- « logs_employe » : Contient les actions des employés.
- « logs_suspendu » : Contient les informations sur les comptes suspendus, notamment pour pouvoir les rétablir.

Déploiement avec Docker (en local) :

En parallèle du déploiement distant, j'ai également mis en place une version conteneurisée du projet via Docker et Docker Compose pour faciliter le déploiement en local avec toutes les dépendances.

J'ai défini :

- Un Dockerfile pour construire une image Apache avec PHP et les extensions nécessaires (pdo_mysql, mysqli, etc.)
- Un fichier docker-compose.yml pour orchestrer l'application web, la base MySQL et le serveur MongoDB
- Un fichier .dockerignore pour exclure les fichiers inutiles lors du build Docker
- Un fichier .htaccess pour permettre au routeur JavaScript de gérer les routes coté client en SPA (Single Page Application).

L'application peut être lancée depuis la commande :

docker-compose up --build

Et le site est accessible à l'adresse : <http://localhost:8080>

Cela permet de mettre en place un environnement de test identique pour tous les postes et simplifie la mise en production sur une plateforme compatible comme Render.com.

Gestion de projet



Image Pixabay

Choix méthodologique :

Pour le développement de l'application EcoRide, j'ai fait le choix d'une organisation agile.

Plus concrètement, j'ai appliqué la méthode Kanban avec des User Stories et une planification visuelle via Trello. Cette méthode m'a permis d'avoir une gestion par priorités, de rester flexible face aux imprévus techniques, de livrer régulièrement des versions fonctionnelles et de suivre efficacement l'évolution du projet.

Evolution du projet :

Mon projet s'est articulé autour de différentes phases. Le mot d'ordre était un développement étape par étape :

1. Analyse des besoins : Donc lecture du sujet, découpage fonctionnel en suivant les User Stories et les besoins, et écriture de ces User Stories dans Trello.
2. Conception : Zoning papier, wireframes et mockups sur Figma, charte graphique sur Figma, réflexion UX/UI, création des diagrammes.
3. Environnement de travail : Installation des outils (XAMPP, MongoDB Compass, VSCode, GitHub, Composer, Docker...).
4. Développement front end : Routage en Javascript, création des pages HTML/CSS/Bootstrap.
5. Développement back end : Modèle en PHP connecté à MySQL et MongoDB, Contrôleur en Javascript avec de l'injection HTML dynamique. Développement dans

un premier temps page par page en suivant les User Stories associés aux pages quand c'était possible.

6. Finalisation des user Stories : Les User Stories qui interconnecte plusieurs pages ont été développée en dernier.
7. Tests : Les tests ont été menée sur chaque fonction écrite et/ou chaque page finalisée. Des comptes variés de profils test ont été utilisées à différentes fin. Une fois l'application fini, un test sur son ensemble a été mené. Suite à ces teste, des corrections et nettoyages ont été menés.
8. Déploiement : Synchronisation de GitHub avec Heroku, vérification des .env, tests en conditions réelles et conteneurisation du projet avec Docker et Docker Compose pour un déploiement en local.
9. Livraison finale : Constitution du documentation technique, manuel d'utilisation, dossier de gestion de projet, charte graphique, dépôt GitHub, base de données, maquettes et Trello.

Tout au long du projet, j'ai également tenu un dossier de suivi au format Word. Ce document regroupe mes réflexions, mes idées, des notions de cours et de code que je souhaitais me remémorer ainsi que l'ensemble du code rédigé étape par étape, accompagné des modifications successives (à la manière d'un système de version comme Git). Il m'a servi de support personnel, à la fois rassurant et utile, notamment pour retrouver une idée, retracer l'évolution d'un module ou revisiter une notion technique

Trello :

La plateforme Trello a été mon point pivot pour la gestion convenable du projet. Je pouvais retrouver chaque tâche effectuer, basculer les cartes dans différentes zones et les faire revenir dans la colonne « Révision du code » en cas de bug.

Mon Trello, construit sous forme Kanban, se présentait ainsi (de gauche à droite) :

- Backlog : Contient toutes les idées, tâches futures et User Stories à traiter.
- Conception : C'est un pense-bête pour les principes à respecter (écologie, cohérence graphique, contraintes du sujet).
- A faire : Tâches prêtes à être développées, écrites le plus souvent en User Stories du type : "En tant que, je veux, afin de".

- En cours : Cartes du Kanban actuellement en développement.
- Révision du code : Fonctionnalités terminées nécessitant des corrections, un refactoring ou une relecture de code.
- Test : Fonctionnalités à tester (données dynamiques, affichage, rôle, correction effectuée à tester, responsive...).
- Terminé (branche développement) : Fonctionnalités prêtes à être livrées mais nécessite une dernière relecture ou une revue de code avant le merge sur la branche stable.
- Terminé (merge branche main) : Fonctionnalités prêtes et validées, intégrées dans la branche principale.
- Conception terminé : Liste les étapes de design finalisées (zoning, wireframes, mockups, charte graphique, logo).
- Environnement de travail installé : Colonne dédiée à la mise en place technique : IDE, extensions, serveur local, accès MongoDB/MySQL, installation Composer, etc.

Précision sur des étapes du développement :

Environnement de travail :

J'ai commencé par installer les outils nécessaires pour un environnement de travail confortable et fonctionnel :

- XAMPP (Apache + MySQL),
- MongoDB Compass pour les logs NoSQL,
- VSCode avec les extensions utiles (Git Graph, MongoDB for VSCode, PHP Intelephense, PHP Namespace Resolver, PHP Unit, PHP Server, Live Sass Compiler),
- Git pour le versionnage,
- Composer pour les dépendances PHP (phpmailer, dotenv, mongodb).
- Docker Desktop (contenant Docker Compose) pour proposer une version conteneurisée du projet

Chaque outil a été validé et suivi dans la colonne dédiée de Trello (carte : « Installer les extensions VSCode » dans la colonne « Environnement de travail installé »).

Phase de conception :

Avant le codage, j'ai réfléchi à la structure globale :

- J'ai réalisé un zoning sur papier puis les wireframes et mockups sur Figma,
- Construction d'une charte graphique écoresponsable avec des couleurs naturelles mais plus sur un ton foncé pour réduire l'utilisation de la batterie. J'ai également utilisé la police intégrée de Bootstrap qui évite de surcharger le code et permet de garder une police populaire ("*Segoe UI*", *Roboto*, "*Helvetica Neue*", *Arial*, sans-serif).
- Rédaction de User Stories centrées sur les besoins des utilisateurs :

"En tant que chauffeur, je souhaite pouvoir appuyer sur le bouton « démarrer » depuis mon espace afin de lancer le covoiturage. "

- Création d'un diagramme de classe via dbdiagram.io,
- Élaboration de diagrammes d'utilisation et de séquence via draw.io pour les parcours

Mise en place du routage JavaScript :

Lors de la conception du projet, j'ai fait le choix d'utiliser PHP vanilla et non un framework comme Symfony ou Laravel. Ce choix était autorisé et je voyais là-dedans, dans le cadre de ma formation, l'occasion d'apprendre l'ensemble des bases de PHP (structure de projet, routage, gestion des formulaires, sessions, sécurité...). Ainsi, j'ai pu mieux comprendre le fonctionnement d'un framework qui automatise certains aspects lors de la programmation.

Cela me permettait d'avoir également plus de contrôle sur le projet et PHP vanilla est plus léger qu'un framework, le projet contenant uniquement ce dont il a besoin.

Dans ce cadre, j'ai développé mon propre système de routage inspiré d'un exercice de travaux pratique abordé dans un cours.

J'ai construit une page socle « index.php » sur lequel se greffe un système de navigation dynamique.

Le routeur identifie les éléments à injecter selon l'attribut id de la balise <main>, ce qui permet de charger les pages de manière fluide sans rechargement complet.

Ce système m'a permis d'implémenter une gestion des droits d'accès et des redirections conditionnelles selon le rôle de l'utilisateur (visiteur, passager, chauffeur, etc.).

Enfin, il a grandement facilité la structuration du code selon le modèle MVC, en séparant clairement les responsabilités entre la Vue, le Modèle et le Contrôleur.

Développement des Vues principales :

J'ai ensuite conçu les pages principales (accueil, inscription, espace personnel, covoiturage...) en HTML/CSS avec Bootstrap 5. La structure est responsive par défaut grâce au système de grille de Bootstrap et l'intégration respecte la charte graphique grâce à une palette de couleurs personnalisée définie en SCSS.

Intégration dynamique grâce au Contrôleur et au Modèle :

Une fois les vues terminées, j'ai repris les User Stories de ma plateforme Trello, qui sont étiquetées par page, afin de développer le contrôleur et le modèle associés, page par page.

Je me suis concentré sur les User Stories n'impliquant pas de logique d'interaction complexe entre plusieurs pages.

Chaque développement suivait un cycle complet avant de passer à la fonctionnalité suivante. Ce cycle était constitué de la création des fichiers JavaScript pour le contrôleur, l'envoi des requêtes fetch vers les fichiers PHP du modèle, la gestion sécurisée des données en MySQL via PDO, et l'enregistrement des logs ou actions spécifiques dans MongoDB pour plus de flexibilité.

Les User Stories plus complexes :

J'ai terminé les User Stories par le développement de ceux impliquant une interconnexion entre plusieurs pages.

Ces fonctionnalités, plus complexes, nécessitaient une coordination entre différentes vues, contrôleurs et modèles. Elles ont donc été finalisées en dernier, une fois que chaque composant isolé était fonctionnel, afin d'assurer une intégration fluide, facile et cohérente dans l'ensemble de l'application.

Les derniers rôles et pages à avoir été développés ont été les employés puis l'administrateur.

Tests :

Pour tester chaque fonctionnalité, j'ai créé des profils de test différents (Passager, Chauffeur, Passager-chauffeur avec des comptes Gmail associés et enfin un compte employé et administrateur).

Par ces profils, j'ai notamment pu tester :

- L'envoi d'emails par PHPMailer avec SMTP sécurisé,
- Vérifier le bon affichage et la bonne récupération des logs MongoDB (avis, suspensions, crédits) même si je pouvais les visualiser dans MongoDB Compass,

- Vérifier également le bon changement et récupération des données sur MySQL, même si je pouvais visualiser ces modifications sur phpMyAdmin,
- Vérifier les limitations d'accès pour chaque rôle.

Déploiement continu avec Git et Heroku :

J'ai utilisé pour ce projet un workflow Git structuré :

- Une branche « main » qui est la branche principale et contient la version stable,
- Une branche « dev » qui contenait la branche en développement.
- Des branches « feature/* » et « bugfix/* » en sous branche de « dev » pour développer des fonctionnalités spécifiques ou réparer des bugs. Je faisais ensuite un merge avec la branche dev, une fois le test effectué.

La base relationnelle est hébergée via JawsDB MySQL et la base NoSQL via MongoDB Atlas.

Les variables d'environnement sont stockées dans .env pour la version locale et inscrit dans .gitignore pour ne jamais être poussée sur le repository. Dans la version déployée, les variables d'environnement sont inscrites dans les « Settings » d'Heroku .

Déploiement local conteneurisé avec Docker :

Dans le cadre de mon examen, j'ai mis en place une version conteneurisée du projet avec Docker et Docker Compose. Ceci permet de faciliter l'installation et garantir un environnement de développement identique sur toutes les machines.

J'ai défini un fichier Dockerfile basé sur une image PHP/Apache avec les modules nécessaires activés (mod_rewrite, headers). Un fichier .htaccess gère les routes pour le fonctionnement SPA côté client, tout en bloquant l'accès aux dossiers sensibles.

Le fichier docker-compose.yml orchestre les services : application web, base MySQL et serveur MongoDB.

Le lancement se fait en une commande (*docker-compose up -d*) pour un environnement prêt à l'emploi, sans configuration manuelle.

Bilan :

Grâce à cette méthode je pouvais me repérer plus aisément dans mon projet, ce qui m'a facilité la tâche pour la rédaction du README.md.

Avoir tenu un journal de bord m'a également rassuré et permit d'approfondir mes connaissances d'une manière plus sereine,

J'ai pu prioriser facilement les tâches grâce à Trello et visualiser rapidement mon parcours me permettant ainsi d'anticiper ma gestion du temps.

Ensuite :

Même si l'application est déployée, coté développement j'ai encore des tâches affichées dans mon trello. Ce sont des tâches en dehors des User Stories mais que j'ai rajouté parce qu'elles me semblent nécessaire pour assurer le bon fonctionnement et l'optimisation de l'application.

Voici plusieurs d'entre elle avec des propositions de solution :

« En tant que Passager, suite à une réservation de plusieurs places pour un trajets, je souhaite laisser un avis pour l'ensemble de mes réservations. »

Solution possible : Ajouter une colonne nb_places_reservees dans la collection MongoDB avis_trajet ou dans la table reservations pour indiquer combien de places concernent l'avis. Cela permettrait à l'utilisateur de laisser un seul avis par trajet, indépendamment du nombre de places réservées.

« En tant qu'Employé, lors de la validation d'un avis concernant plusieurs réservations pour un trajet, je souhaite que les crédits soient attribués en équivalence avec le nombre de places réservés. »

Solution possible : Lors de la validation de l'avis par l'employé, le script PHP gestion_avis.php devra inclure une récupération du nombre de places réservées et appliquer le crédit en fonction de ce nombre. Exemple : prix * nb_places.

« En tant qu'Employé, je souhaite voir sur un tableau l'ensemble des avis refusé ou validé. »

Solution possible : Ajouter une interface avec des filtres (statut = validé/refusé) et interroger MongoDB pour afficher tous les avis selon leur statut.

« Lors de la création d'un compte, il faudrait que la note initiale soit à 0 et non à NULL. »

Solutions possibles : Ajouter une valeur par défaut (0) dans la base de données ou l'ajouter explicitement dans le script inscription.php.

« En tant qu'Utilisateur, je souhaite un bouton pour supprimer mon compte. »

Solution possible : Ajouter un bouton "Supprimer mon compte" dans l'espace utilisateur, qui envoie une requête vers un fichier `supprimer_compte.php`. Celui-ci pourra :

- désactiver le compte (note = -1) ou
- supprimer toutes les données associées après confirmation de l'utilisateur.

« En tant que Passager, je souhaite être remboursé entièrement si mon trajet est annulé »

Solution possible : Lorsque le trajet passe à l'état annulé, un script pourrait récupérer tous les passagers concernés, recréditer leur compte (selon le prix * nb de places), et enregistrer un log MongoDB `remboursement_annulation`.

« En tant qu'Utilisateur, je souhaite avoir une page « Erreur 404 » plus user friendly. »

Solution possible : Créer une page `404.html` avec un visuel (icône, illustration SVG), un message clair ("Oups ! Page introuvable") et un bouton de retour vers l'accueil.

« En tant qu'Utilisateur, je souhaite pouvoir recevoir par mail une réinitialisation du mot de passe sur la page de connexion en cas d'oubli. »

Solution possible : Ajouter un lien « Mot de passe oublié ? » sur la page de connexion, qui redirige vers un formulaire demandant l'adresse email de l'utilisateur.

À la soumission un token unique est généré et enregistré avec une date d'expiration et un mail est envoyé avec un lien.

« En tant qu'Utilisateur, je souhaite pouvoir avoir un visuel convivial et plus fonctionnel sur le smartphone. »

Solution possible : Intégration responsive via Bootstrap 5 (grille `col-12 col-md-6`, classes `d-flex`, `text-center`, etc.), design mobile-first avec `@media` personnalisés dans `app.scss`. Les composants sont adaptés pour le tactile (boutons larges, textes lisibles, espacement augmenté).

« En tant qu'Utilisateur, je souhaite pouvoir recharger mon compte en crédit. »

Solution possible : Ajouter une page "Rechargement" avec choix du montant et paiement sécurisé (type Stripe). Une fois validé, incrémenter les crédits dans la base MySQL et enregistrer un log de rechargement dans MongoDB (logs_rechargement).

« En tant qu'Utilisateur chauffeur, je ne dois pas avoir accès à la page « Covoiturage »

Solution possible : Dans allRoutes.js, j'attribue les droits d'accès par page et dans navbar.js je peux cacher l'onglet « Covoiturage » si l'utilisateur qui se connecte est un chauffeur.

« En tant qu'Utilisateur, je veux autoriser l'utilisation de cookies lors de la connexion. »

Solution possible : Suite à la connexion, afficher un message javascript invitant à valider l'autorisation de cookies.

« En tant qu'Utilisateur, je veux que le site soit optimisé pour les malvoyants. »

Solution possible : Vérifier et rajouter la balise « alt » avec un descriptif pour chaque image.

« En tant qu'Utilisateur, je veux pouvoir effectuer une recherche depuis la page d'accueil.»

Solution possible : Connecter le formulaire de la page d'accueil avec covoit.js et suite à une recherche effectuer une redirection vers la page Covoiturages.

Charte graphique



Image Pixabay

Couleurs et police :

J'ai pensé ma charte graphique pour avoir des tons sombres et ainsi limiter la consommation d'énergie des appareils.

Les couleurs vont se situer dans le gris, marron, vert, beige pour rappeler des couleurs naturels hormis les étoiles représentant les notes des chauffeurs que j'ai souhaité garder jaune.

Voici mes codes couleurs :

Primary: #A16D24;	
Secondary: #C5B585;	
Background: #CFA549;	
Success: #1C6F21;	
Beige: #E3D2AE;	
Star: #F39A0A;	

Concernant la police, j'ai utilisé celle intégrée par défaut à Bootstrap (*Segoe UI, Roboto, Helvetica Neue, Arial, sans-serif*).

Ce choix permet de limiter le chargement de polices externes, ce qui allège le code, améliore les performances, et participe indirectement à une consommation énergétique réduite, tout en assurant une lisibilité optimale sur la plupart des appareils.

Wireframes et Mockups :

Dans le cadre de l'examen, 3 wireframes et 3 mockups seront présentés pour la version bureautique et mobile. Ils seront intégrés en PDF dans les pages suivantes.

Le visuel présentera la page Accueil, Covoiturage et Espace Administrateur.

L'ordre de présentation sera :

- Wireframe bureautique de la page d'accueil
- Mockup bureautique de la page d'accueil

- Wireframe bureautique de la page covoiturage
- Mockup bureautique de la page covoiturage

- Wireframe bureautique de la page espace administrateur
- Mockup bureautique de la page espace administrateur

- Wireframe mobile de la page d'accueil
- Mockup mobile de la page d'accueil

- Wireframe mobile de la page covoiturage
- Mockup mobile de la page covoiturage

- Wireframe mobile de la page espace administrateur
- Mockup mobile de la page espace administrateur